

2026년 6월 3일

基于动态负载均衡策略优化的分布式系统性能研究

증여자:
YINJUNLONG
50250335

목차

01

연구 배경 및 문제점

02

동적 부하 분산 모델 설계

03

실험 환경 및 실험 코드

04

실험 결과 및 성능 분석

01

研究背景与问题

연구 배경: 정적 부하 분산의 병목 현상



정적 폴링의 근본적인 결함

요청은 고정된 순서로 할당되며 CPU 부하, 메모리 사용량, 활성 연결 수와 같은 동적 지표는 완전히 무시되므로 노드의 실시간 상태를 파악하는 것이 불가능합니다.



일반적인 결과

일부 노드는 CPU 사용률이 90%를 초과하는 반면, 다른 노드는 35% 미만의 유휴 상태이며, 부하 변동률이 최대 156.4에 달해 자원 활용에 심각한 불균형이 있음을 나타냅니다.



현실적인 문제들이 더욱 심화되고 있습니다

컨테이너 기반 및 마이크로서비스 아키텍처에서는 노드 이질성, 탄력적 확장, 갑작스러운 트래픽 급증이 점점 더 흔해지고 있어 정적인 전략이 효과적이지 못합니다.



실시간 상태 파악

동적 전략은 노드 성능 지표를 지속적으로 수집하고 시스템 상태 변화에 밀리초 단위로 대응하여 핫 노드의 과부하를 방지합니다.



적응형 교통 스케줄링

요청은 부하 차이에 따라 지능적으로 할당되어 클러스터 리소스를 균형 있게 사용하고 부하 변동 및 응답 지연 시간을 크게 줄입니다.



유연한 확장 및 수축 적응

컨테이너화 및 마이크로서비스 아키텍처를 원활하게 지원하고, 동적인 노드 변경에 자동으로 적응하며, 높은 서비스 가용성을 보장합니다.

실시간으로 시스템 상태 변화를 감지하고 대응할 수 있는 동적 부하 분산 전략은 학계와 산업계 모두에서 주목받고 있습니다.

연구 질문 및 가설

RQ1

중대형 동시 접속 환경에서 동적 전략은 처리량을 크게 향상시킬 수 있을까요?

RQ2

동적 전략이 평균 응답 시간과 P95 테일 지연 시간을 효과적으로 줄일 수 있을까요?

RQ3

동적 스케줄링으로 인해 발생하는 추가적인 자원 오버헤드가 프로젝트에 허용 가능한 범위 내에 있습니까?

H1

동시 접속 사용자가 500명 이상일 경우, 정적 풀링 방식에 비해 처리량이 최소 20% 향상됩니다.

H2

모든 동시 접속 수준에서 평균 응답 시간이 최소 15% 감소합니다.

H3

추가 CPU 오버헤드는 10%를 초과해서는 안 되며, 추가 네트워크 트래픽은 5%를 초과해서는 안 됩니다.

검증 기준

 정량화 임계값: 통계적 유의성을 확보하려면 성과 지표가 미리 설정된 백분율 개선을 달성해야 합니다.

 공학적 제약 조건: 최적화의 부작용이 이점을 능가하지 않도록 자원 비용은 허용 가능한 범위 내로 유지되어야 합니다.

3

연구 질문

3

가설 검증

02

动态负载均衡模型设计

시스템 아키텍처 및 부하 평가 모델



3단계 아키텍처 설계

- 0 클라이언트 계층: JMeter는 실제 사용자 요청을 시뮬레이션하여 스트레스 테스트의 진입점 역할을 합니다. 로
- 1 드 밸런싱 계층: NLS 점수를 기반으로 요청을 동적으로 할당하는 스케줄링 코어입니다. 백엔드
- 2 서비스
- 0 계층: 실제 비즈니스 처리를 담당하는 3~5개 노드로 구성된 탄력적인 클러스터입니다.
- 3



엔로딩 정책: NLS > 0.85일 때 자동으로 트리거되며, 새로운 요청을 정상적인 노드로 라우팅합니다.

NLS 점수 산정 지표 가중치: 가중치 선택 기준

CPU 사용률 0.35% 는 CPU 포화 상태를 나타내며, 이는 처리량 감소의 가장 직접적인 원인입니다.

메모리 사용량이 0.25% 만 되어도 너무 높아 스왑이 발생하고 응답 시간이 급격히 저하됩니다.

활성 연결 비율 이 0.20이라는 것은 네트워크 I/O 부하와 큐 포화 상태를 반영합니다.

응답 시간 비율 0.20은 노드의 서비스 처리 속도를 직접적으로 측정합니다. 노드 부하 점수(NLS)

$$NLS = 0.35 \times \text{cpu} + 0.25 \times \text{mem} + 0.20 \times \text{conn} + 0.20 \times \text{resp_time}$$

NLS 점수가 낮은 노드일수록 새로운 요청을 수신하기에 더 적합합니다. NLS 점수가 0.85보다 크면 로드 밸런싱이 작동합니다.

03

实验环境与实验代码

실험 환경 구성

프로젝트	구성 내용
클러스터 크기	로드 밸런싱 노드 1개 + 백엔드 서비스 노드 3~5개
로드 밸런싱 노드	쿼드코어 CPU, 8GB RAM, 우분투 22.04
백엔드 서비스 노드	쿼드코어 CPU, 16GB RAM, SSD, 우분투 22.04
네트워크 환경	기가비트 이더넷, 노드 간 지연 시간 < 0.2ms
응용 환경	자바 17, 스프링 부트 3.1, Nginx 1.24
스트레스 테스트 도구	Apache JMeter 5.6
모니터링 도구	프로메테우스 2.47 그라파나 10.0
모니터링 구성 요소 배포	실험 결과에 영향을 주지 않도록 전용 노드(2코어 CPU, 4GB RAM)에 독립적으로 배포했습니다. Prometheus 데이터 수집 간격은 10초이며, Grafana를 사용하여 데이터를 정리합니다. 새로운 간격 5초

작업 부하 설계 및 데이터 수집

단계별 동시 부하 설계

동시 사용
자

장면 배치 예상 목표

100가지 저부하 시나리오

저부하 조건에서 동적 전략에 허용할 수 없는 손실이
발생하지 않는지 확인하십시오.

500 중부하 동적 전략의 장점이 드러나기 시작하고 있습니다.

1000개의 고부하 스케줄링을 통해 얻을 수 있는 상당한 이점

2000년 극한 하중 조건에서 동적 전략의 이점을 극대화하기

2분

등반 기간

8분

정상 상태 획득

3번

반복적으로 평균을 구하세요



실험적 통제

2분간의 시스템 부하 증가 기간과 그 후 8분간의 정상 상태 데이터 수집을 통해 시스템이 안정적인 상태에 도달하도록 합니다.

각 그룹은 독립적으로 세 번 반복되었으며, 무작위 변동의 영향을 제거하기 위해 평균값을 사용했습니다.



2차원 데이터 수집

클라이언트 측: JMeter는 TPS(초당 트랜잭션 수), 응답 시간 및 오류율을 수
집합니다. 서버 측: Prometheus와 Micrometer는 CPU, 메모리 및 네트워크를 모니터링합니다.
SWC



재시작 절차

과거의 국가적 간섭을 제거하기 위해 전략을 바꾸기 전에 프로세스를 다시 시작하십시오.

기준선 일관성을 확보하기 위해 CPU 사용률이 10% 미만으로 떨어질 때까지 기다리세요 .

스케줄링 알고리즘 구현 설명

1

CPU, 메모리, 연결, 반응 속도를 포함한 네 가지 지표가 500ms마다 수집됩니다.



2

표준화

[0,1] 구간으로의 최소-최대 매핑은 차원 차이를 제거합니다.



3

하중 등급

가중 합산 방식을 사용하여 NLS를 계산하고, 점수표는 동적으로 업데이트됩니다.



4

일정 결정

가장 낮은 NLS 노드를 선택하고, 값이 0.85보다 크면 오프로드를 트리거합니다.



동시성 안전성 평가표

는 Concurrent HashMap을 사용하여 데이터 수집 코루틴과 스케줄링 메인 스레드의 동시성 안전성을 보장합니다.



분리형 설계 란 메트

릭 수집이 독립적인 백그라운드 코루틴에 의해 주기적으로 트리거되어 요청 전달 경로와 분리되고 수집 지연이 요청 지연 시간에 미치는 영향을 방지하는 것을 의미합니다.

04

实验结果与性能分析

처리량 및 응답 시간 결과

처리량 비교 (TPS)

69.4%

처리량 개선율

2000 동시 접속

50.3%

평균 지연 시간 감소

↓ 50.3%

53.5%

P95 지연 시간 감소

↓ 53.5%

1.7배

수익성

높은 동시성

동적 전략의 비선형 성장 특성은 동시



접속 규모가 증가함에 따라 수익률이 비선형적으로 증가함을 보여줍니다. 동시 접속 100명일 때 수익률은 8.2%에서 2000명일 때 69.4%로 증가하며, 특히 동시 접속 규모가 매우 큰 시나리오에서 수익률이 크게 증폭됩니다.

최대 2000명의 동시 사용자가 접



속하는 극한 부하 조건에서 동적 로드 밸런싱은 처리량을 69.4% 증가시키면서 P95 지연 시간을 53.5% 감소시켜 고부하 환경에서도 뛰어난 안정성을 입증했습니다.

자원 지출 및 통계 테스트

+6%ⁱ

CPU 사용률 차이

H3는 만족합니다

+4.2%

추가 네트워크 트래픽

H3는 만족합니다

-62%

오류율 개선

상당한 개선

노드 부하 분산 성능 (동시 접속 CPU 2000개 분산)

전략	노드 1	노드 2	노드 3	노드 4	부하 분산
정적 풀링	91%	38%	85%	32%	156.4
동적 평형	62%	58%	65%	60%	23.8

가설 검정 통과

세 가지 가설 H1/H2/H3 모두 t-검정을 통과했습니다($p < 0.05$) .
(0.05), 따라서 통계적 유의성이 확인되었습니다.

탁월한 성능 향상

부하 변동률은 84.8% 감소했으며, 성능 향상이 추가 비용을 상회했습니다.
벨기에 10:1

끝

感谢聆听